Interpreting Index Scan Execution Plan Info

One of the main use cases for Indexes is efficient lookup and filtering. When a column `user_id` is listed in a `WHERE` clause, make sure there's an index for the table covering `user_id` either as a single column index, or as part of a Multicolumn index where `user_id` is the first column listed.

PostgreSQL won't always choose to use your index until row counts are high, and when a single row or a small set of rows are being requested. PostgreSQL uses the selectivity of the query, cardinality for the column, and compares the costs between scanning the whole table, fetching from the index and table, or using the index alone to determine what's best.

When the index is used, the planner shows an *Index Scan* and lists the name of the index.

The basics of index data structure internals are as follows. Each item in the Index is a reference to a table row. Scanning an Index is typically a faster way to collect a list of row pointers compared with scanning all rows for a large table.

Table access is slower because the data is spread out in random locations. Indexes, on the other hand, are ordered and designed for fast retrieval. Because of this ordered storage characteristic, besides filtering, Indexes can also be used for sorting with `ORDER BY` operations.

The values of a node within the index include a `ctid`. The `ctid` is a pointer to the disk location of the row data. You can add the `ctid` to your query to see the value.

Besides Index Scans, there is an even more efficient scan type related to indexes.

With *Index Only Scans*, PostgreSQL fetches all of the data it needs from the index itself directly without needing to access the heap where the rows are.

Index Only Scans are not supported for all index types. B-tree indexes always support Index Only Scans. Typically B-tree indexes are used for equality comparisons such as comparing the exact value of `user_id = 1`. An index covering all values of `user_id` will be traversed until the value is found.

For now, Indexes can be put into two major categories. One category is a general purpose index and the second category would be a "specialized" index. A general purpose index is useful for many queries and operation types while a specialized index might be designed to be optimal for one query.